

Exact-Oracle Post-Placement Refinement for Multi-Bit Flip-Flop Banking: Structural Rebanking without Cascades

Anonymous Author(s)
Anonymous Institution

Abstract—Multi-bit flip-flop (MBFF) banking trades clock power and area against timing, and is typically driven by proxy cost models whose mispricing compounds: each committed merge silently distorts the cost of every later decision. On the ICCAD 2024 CAD Contest Problem B benchmark, we show that this pricing error, not the move class, is the root cause: structural rebanking moves that catastrophically cascade under a one-hop proxy model (up to +254% score) become strictly profitable when re-priced exactly. We present a post-placement refinement framework built on an evaluator-exact, side-effect-free incremental timing oracle that prices every candidate move—bit re-pairing, structural rebanking ($2 \times 2b \rightarrow 4b$, $4 \times 1b \rightarrow 4b$), positional relocation and swaps, and bin-density eviction—in closed loop with the official cost function. Exact pricing enables (i) batch local search that safely commits an order of magnitude more moves per round than conservative conflict-free batching, (ii) monotone-accept structural rebanking with rollback-safe staging, and (iii) refinement-side pricing of the density penalty term. On all seven contest benchmarks, a single configuration of our flow beats the contest top-3 best entry per case, the first published method to do so, reaching a composite ratio of 0.953, where the strongest previously published methods reach 0.979–0.988, with every solution verified legal by the official evaluator.

Index Terms—multi-bit flip-flop, banking, incremental timing analysis, detailed placement, local search

I. INTRODUCTION

Multi-bit flip-flop (MBFF) banking replaces groups of single-bit registers with multi-bit trays that share internal clock inverters, cutting clock dynamic power and cell area at the price of placement freedom and timing slack. Post-placement banking has a long lineage, from power-driven merging under placement and timing constraints [1] and interval-graph formulations [2] to flop-tray implementation in commercial-quality flows [3]. The ICCAD 2024 CAD Contest Problem B [4] distilled the problem into a reproducible benchmark: banking and placement are optimized jointly under a four-term cost $\alpha \cdot TNS + \beta \cdot \text{Power} + \gamma \cdot \text{Area} + \lambda \cdot \text{Density}$, scored by an official evaluator on seven industrial-scale cases with hard legality constraints; the 2024 contest drew 221 registered teams across its three problems [5].

Banking is a sequence of committed discrete decisions, and nearly every published flow prices those decisions with a proxy timing model. Proxy error does not average out. Each mispriced merge is committed, physically moves pins, and distorts the slack landscape that prices every later decision. We measured this compounding directly on the contest benchmark. Our baseline, a top-ranked open-source implementation of the

contest reference flow, carries a one-hop timing proxy that underestimates evaluator TNS by roughly 34% on the hardest case. Under that proxy, cascades of mispriced $2 \times 2b \rightarrow 4b$ merges regress the score by up to +254%, and a debank-and-rebank pass by +134% (Sec. II). The natural reading is that structural rebanking after legalization is too dangerous a move class. Our central claim is the opposite: under exact per-move pricing with staged, monotone acceptance, the same move class becomes strictly profitable; a controlled counterfactual (Sec. V-C) puts the price of proxy quotes at up to 0.9% of final score. Pricing, not the move class, is the blocker.

We therefore rebuild the refinement stage around exact pricing. The upstream flow builds on the open-source baseline (debanking, slack-driven global placement, clustering-based banking, legalization, detailed placement), which we had previously strengthened under the same proxy pricing (Sec. II); those upstream enhancements are not claims of this paper. Everything downstream of detailed placement is the contribution here:

- **An evaluator-exact, side-effect-free incremental timing oracle.** It prices any candidate move (bit re-pairing, structural rebanking, positional relocation and swaps, and bin-density eviction) against the official cost function without mutating placement state. Incremental and full recomputation agree to a difference of 0.0 in every round, and the side-effect-free delta matches a real apply-and-revert to within 7.4×10^{-13} .
- **Exactly re-priced batch local search.** Conflict-free batching, the standard safety device in batched placement moves, commits about 10% of improving candidates per round (152 of 1,703 on one hidden case). Because every candidate is re-priced exactly against the committed state at apply time, the disjointness requirement can be dropped entirely: a stale quote can only cause a rejection, never a bad commit.
- **Monotone-accept structural rebanking with rollback staging.** Merges ($2 \times 2b \rightarrow 4b$, $4 \times 1b \rightarrow 4b$) are staged outside the committed netlist, priced exactly, committed only on strict improvement, and rolled back completely otherwise.
- **Density-term pricing combined with exact timing pricing.** A repair pass evicts flip-flops from violating bins and accepts an eviction chain only if the exact

timing delta plus the density delta improves the total cost, making all four cost terms visible to refinement.

The rebuild also removed a silent move-space restriction in the baseline: a work-queue flag misread as placement liveness excluded 32% of flip-flops from every refinement operator (2,024 of 6,309 on one hidden case, including all 2,000 of its two-bit MBFFs).

Figure 1 summarizes the framework. A single configuration, unchanged across all seven benchmarks, beats the best contest top-3 entry on every case, the first published method to do so; it also beats every one of our per-case-tuned records. The composite score ratio against the contest top-3 is 0.953, versus 0.991 for the contest winner’s method under the same baseline (self-reported 0.988 against first place; published as a two-page late-breaking result [6]); ours is the first full-length method paper at this quality level on this benchmark (TIMBER [7] reports strong runtime and memory results on the same cases but is not evaluator-comparable; Sec. V-B). Every solution we report passes the official evaluator and both legality checkers.

II. BACKGROUND AND MOTIVATION

A. Problem B and the Official Evaluator

The ICCAD 2024 CAD Contest Problem B provides a legalized design containing single- and multi-bit flip-flops, fixed combinational gates, and a cell library with MBFF variants of differing bit widths, power, area, and pin geometry. A solution rebanks and replaces the flip-flops to minimize

$$S = \alpha \cdot TNS + \beta \cdot Power + \gamma \cdot Area + \lambda \cdot V, \quad (1)$$

where V counts placement bins whose utilization exceeds a per-testcase threshold, subject to hard legality constraints: no overlaps, and all cells on rows inside the die [4], [5]. The weights $\alpha, \beta, \gamma, \lambda$ and a *DisplacementDelay* constant vary per testcase, which shifts the dominant cost term between benchmarks: some cases are priced almost entirely on *TNS*, others on power and area.

Scoring is defined operationally by an official evaluator binary, and its timing semantics differ from conventional net models. The delay of each timing arc is *DisplacementDelay* times the two-point half-perimeter wirelength between the driver pin and that individual sink pin; there is no shared net bounding box and no Steiner topology. Each sink’s slack is the base slack given at parse time, adjusted by the change in the driving flip-flop’s Q-pin delay when its cell type changes and by the change in the wire delays of the affected arcs. *TNS* sums the negative D-pin slacks over all flip-flops. Any refinement engine that aspires to monotone score descent must reproduce these semantics exactly, because Eq. (1) is evaluated on them and on nothing else.

Our flow builds on a top-ranked open-source implementation of the contest reference flow (Fig. 1): debank all MBFFs into single-bit registers, spread them with a slack-driven conjugate-gradient global placement, propose spatial groups by mean-shift clustering, bank, then legalize onto rows and run detailed placement. Prior to this work we had

strengthened several of its stages, most notably replacing the greedy banking step with maximum-weight-matching pairing under proximity- and risk-weighted edge costs and adding cost-aware detailed-placement repairs. Those enhancements share the baseline’s one-hop proxy pricing throughout and are not claimed as contributions here; they matter only as the starting point, and they make every refinement delta in Sec. V conservative, since the refinement layer is measured on top of the strengthened rather than the stock front end. Everything from the incremental oracle onward (Secs. III–IV) is the contribution of this paper.

B. Why Proxy Cost Models Diverge

Refinement explores millions of candidate moves per benchmark, so invoking the evaluator per candidate is infeasible and contest-style flows substitute an internal proxy. The proxy in our inherited baseline is representative: it reads each flip-flop’s stored slack and re-prices only one hop of wiring around the moved cell, propagating nothing forward. When a flip-flop moves, the wire delay to every fanout sink changes, which changes the slack of downstream flip-flops, yet the proxy leaves their stored slacks untouched. Each committed move therefore deposits a small pricing error into the design state, subsequent decisions are made against the corrupted state, and the error compounds: on the hardest benchmark in the suite, the internal one-hop model ends the flow reporting a *TNS* roughly 34% below the value implied by the evaluator’s final score decomposition.

The practical damage shows up when the move class gets aggressive. Two prior structural rebanking attempts in the same codebase, one matching flip-flops into higher-bit cells and one debanking then rebanking mispriced groups, degraded the final score by +254% and +134% on a power-dominated hidden case. Both generated exactly the move class our method later exploits profitably: merging smaller flip-flops into 4-bit cells. The moves were sound; the one-hop prices were not, and a single mispriced merge licensed a chain of further mispriced merges. This cascade catalog is the motivating evidence for this paper. It predicts that replacing the proxy with an evaluator-exact oracle, while keeping the move class, converts the cascade into strict descent, and Sec. V confirms the prediction.

C. Related Work

MBFF banking has a long post-placement lineage: merging flip-flops under timing constraints [1], interval-graph formulations [2], flop-tray co-design [3], and capacitated k -means clustering at scale [8]. Chen et al. [9] redistribute positive slack along register chains to enlarge feasible clustering regions before banking; despite the shared term, that is a pre-clustering slack-budget transformation, whereas we redistribute nothing and instead re-price every move exactly after placement.

On the Problem B benchmark itself, the contest winner’s method appears as a two-page late-breaking result [6] with a self-reported average ratio of 0.988 against the first-place entry; the format precludes a full method description, and to

our knowledge no full-length paper has matched that level on this benchmark. Chen et al. [10] adopt a banking-then-refinement flow shape close to ours but price moves with proxy models; TIMBER [7] reports on the same testcases but is not evaluator-comparable (Sec. V-B). Gong et al. [11] target MBFF timing and power at advanced nodes without a public evaluator.

Timing-driven placement attacks the same objective from the placer side [12], [13], [14], but optimizes proxy timing signals rather than closing the loop with the scoring function. Incremental STA engines such as OpenTimer [15], [16] maintain full-netlist timing under general delay models; our oracle is deliberately narrower, reproducing the evaluator’s two-point delay model exactly with side-effect-free deltas cheap enough for inner-loop pricing. Density has been handled analytically in global placement [17], [18], but the contest’s bin-violation term $\lambda \cdot V$ had no published refinement-side pricing combined with exact timing; we price it per move alongside *TNS*.

III. AN EVALUATOR-EXACT INCREMENTAL TIMING ORACLE

A. Design Goal

The oracle prices any candidate move under exactly the timing model the official evaluator applies to the final solution: per-sink displacement delay proportional to the two-point HPWL between driver pin and sink pin, under a single ideal clock [4]. Three properties are required simultaneously. First, *exactness*: the reported ΔTNS must equal the difference the evaluator itself would measure between the pre-move and post-move solutions. Second, *side-effect freedom*: evaluating a candidate must not mutate netlist or timing state, so that thousands of candidates can be screened against one committed state without apply–revert churn. Third, *locality*: the cost of one evaluation must scale with the affected timing cone, not with design size.

General-purpose incremental STA engines meet the second and third goals for full industrial timing [15], [16], but the contest model is far simpler, and that simplicity cuts both ways. It permits closed-form per-sink slack recomputation at cone cost, which we exploit. It also makes cheap proxies tempting: the inherited flow prices moves through a one-hop slack approximation, the DATE 2026 flow states that it estimates timing through a local-critical-path wirelength proxy [10], and the two-page format of [6] leaves its pricing model unspecified; none closes the loop with the scoring function. On the hardest benchmark that proxy underestimates *TNS* by roughly 34%, and its mispricing is what makes structural rebanking cascade (Sec. II). The oracle removes the model error entirely rather than tightening the approximation.

B. Timing State Keyed by Logical Objects

Banking creates and destroys physical cells, so any timing cache keyed by physical instances is invalidated by exactly the moves for which accurate pricing matters most.

The oracle instead keys all topology caches by *immutable logical objects*: the single-bit flip-flops of the debanked netlist

and the combinational gates. Neither set changes during banking; a merge only remaps logical bits to a new (cell, slot) pair. Fanin and fanout cone structure over these logical objects is computed once and survives every structural mutation. Mutable leaf attributes are never cached: a bit’s D/Q pin coordinates and its Q-pin (clock-to-Q) delay are read live through the logical→physical indirection at evaluation time, logical bit → hosting cell and slot → pin offset from the cell’s library entry plus the cell’s current position. Structural moves thus reduce to updating one indirection entry, and no cache ever needs rebuilding.

C. Three Evaluation Modes

The oracle exposes three entry points with a shared cone-propagation core.

Full pass. Recompute every sink slack from scratch in one parallel sweep; the validation ground truth, linear in design size.

Incremental cone recompute. After a move commits, repropagate only the timing cones reachable from the touched pins and fold the slack updates into the committed caches. This keeps the committed state exact between moves at cone cost.

Side-effect-free delta. Price a hypothetical move without touching committed state:

- 1) Build a local *override map* holding, for each moved bit, its hypothetical D/Q pin positions and Q-pin delay.
- 2) Collect the *affected-sink set*: the moved bits themselves, their direct fanout sinks, and the sinks driven through gates in the moved bits’ forward combinational cones.
- 3) Walk the forward cones, resolving every pin attribute through the override map first and the committed state otherwise, accumulating recomputed slacks into thread-local scratch.
- 4) Return the weighted delta $\alpha \cdot \Delta TNS$ from the affected sinks; all scratch is discarded.

Because committed state is read-only during step 3 and scratch is thread-local, arbitrarily many candidates can be priced concurrently against the same committed state. All refinement operators in Section IV use this mode for parallel candidate screening and re-price serially at apply time.

D. Exactness Validation

Two invariants are checked continuously in validation builds. First, after every refinement round the incrementally maintained *TNS* total is compared against a full-pass recomputation; the difference is 0.000000 on every round of every benchmark. Second, each side-effect-free delta is compared against the measured delta of actually applying and reverting the same move; the maximum observed discrepancy is 7.4×10^{-13} , attributable to floating-point summation order. The oracle is therefore not a tighter estimate but an implementation of the declared scoring semantics, self-consistent to floating-point precision. Externally, its maintained totals track the official evaluator within 0.05% (median) across stage-boundary checkpoints on all seven cases, with a maximum residual of 1.1% on the most *TNS*-weighted states; every

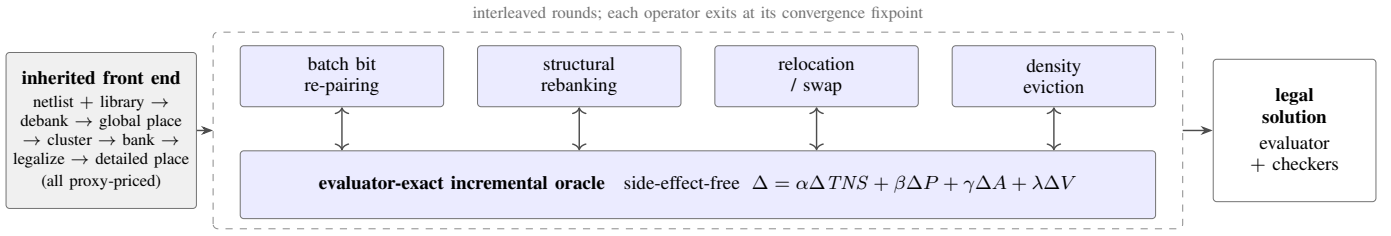


Fig. 1. Framework overview. The front end produces a legal, proxy-priced placement; every downstream accept is priced by the oracle through one interface (a side-effect-free weighted delta); operators run in interleaved rounds to convergence; results are verified by the official evaluator and both checkers.

reported result is in any case scored by the evaluator binary itself (Sec. V-B), so no claim in this paper rests on the residual.

E. From Bit Swaps to Structural Remaps

The delta evaluator was first built for same-cell bit swaps, where two bits exchange slots and only pin positions change. The override map generalizes it at no additional cost: remapping a *group* of bits to a new cell adds exactly one field, the Q-pin delay override taken from the target cell’s library entry, since a different cell type changes clock-to-Q delay as well as pin geometry. One mechanism therefore prices the entire move spectrum—slot swaps, relocations, and group-to-new-cell structural merges—which enables the oracle-priced rebanking of Section IV to screen structural candidates exactly before any netlist mutation occurs.

Exactness here is defined with respect to the benchmark’s declared cost model. Nothing in the framework depends on that model’s simplicity: the operators of Section IV consume the oracle through a single interface, a priced Δ for a hypothetical move, so an incremental signoff-grade engine [16] can substitute for it under richer delay models, trading some pricing fidelity for generality. What this paper isolates, by making the pricing literally exact, is how much solution quality that fidelity is worth.

IV. EXACTLY PRICED REFINEMENT OPERATORS

The oracle of Section III turns refinement into a pricing problem: any candidate move can be quoted its exact change in official cost before it is committed. This section describes the refinement engine built on that capability: three exactly-priced move classes—bit re-pairing, structural rebanking, and bin-density eviction—plus two positional operators inherited from the reference flow (relocation and critical-pair swap), all orchestrated by the interleaved framework of Section IV-D. The three exactly-priced move classes share one contract. A candidate is screened with a side-effect-free delta, re-priced exactly against the committed state at apply time, and accepted only if the full weighted delta $\alpha \Delta TNS + \beta \Delta P + \gamma \Delta A + \lambda \Delta V$ is negative; every committed sequence of these moves is monotone decreasing in the official cost. The two positional operators are scoped more narrowly: they accept on the exact TNS delta at fixed power and area, without pricing the bin term, so V can drift during relocation; the density stage that closes the schedule re-prices every remaining violation explicitly.

A. Batch Bit Re-Pairing

The finest-grained move exchanges individual bits between two multi-bit cells, or permutes bit-to-slot assignments within one cell. The search is a batched best-improvement local search in the spirit of multi-move-per-round methods from scheduling [19] and batched placement refinement: each round enumerates the best swap for every cell in a candidate neighborhood, then commits many improving swaps together. Unlike dynasearch proper, no independence structure between moves is required, because each commit is re-priced exactly (below). Candidate evaluation is embarrassingly parallel because the screening delta touches no shared mutable state; threads price disjoint candidate lists against a frozen snapshot of the committed placement.

Exact pricing changes what a batch may contain. Conventional batched local search commits only conflict-disjoint candidates, since overlapping timing cones invalidate screened gains; under exact re-pricing each survivor is re-quoted against the committed state immediately before it is applied, so a stale gain can only cause a rejection, never a bad commit. The inherited flow’s conservative disjoint batching committed roughly 10% of improving candidates per round (152 of 1,703 on one hidden case). Dropping the disjointness filter commits every candidate that remains improving at apply time, an order-of-magnitude increase in per-round throughput at identical safety.

Intra-cell slot permutation complements inter-cell exchange. Because the evaluator prices each sink pin’s displacement individually, permuting which bit occupies which physical slot of a multi-bit cell changes timing cost without moving the cell. These permutation moves are priced by the same delta evaluator and folded into the same batches.

Incremental engines converge to local optimality only over the candidates they have seen. Periodic full rescans re-enumerate the entire move space and reseed the candidate queue, folding exhaustive search into the incremental engine: between rescans the engine drains cheap dirty-set candidates, and each rescan certifies that no improving move was orphaned by earlier commits. The engine’s exactness is validated continuously under the invariants of Sec. III-D.

B. Oracle-Guided Structural Rebanking

Structural rebanking changes the banking solution itself after legalization, merging $2 \times 2b \rightarrow 4b$ and $4 \times 1b \rightarrow 4b$. Can-

didates are enumerated per cell from its same-clock k -nearest neighbors, so enumeration cost stays linear in design size.

Each candidate passes through two phases. A parallel screening phase prices the merge with the group-move oracle at a centroid placement estimate, evaluating the full objective $\alpha \Delta TNS + \beta \Delta P + \gamma \Delta A + \lambda \Delta V$; power and area deltas are exact from the cell library, the density delta is exact from the bin table, and the timing delta assumes the merged cell lands at the candidates' centroid. Screening is side-effect free, so all candidates are priced concurrently. An exact phase then processes survivors in best-first order of screened gain. The merge is staged rather than applied: netlist map mutations are deferred, a legal site is sought for the merged cell, and the oracle re-prices the move at the actual site. If the exact delta is negative the staged state is committed; otherwise rollback restores wiring, pin mapping, and placement exactly. Acceptance is strictly monotone, and a patience cutoff terminates the phase after a fixed number of consecutive rejections.

The same move class cascades catastrophically under one-hop proxy pricing (Sec. II). Under exact pricing the identical move class produces a strictly monotone descent; on one hidden case each $2 \times 2b \rightarrow 4b$ merge alone is worth $-3,000$ weighted power-and-area units. The difference between the cascading and the profitable outcomes lies in the accept pricing and in the staged commit structure that exact pricing makes safe to relax; Sec. V-C separates the two contributions experimentally.

C. Density-Term Pricing

The density penalty $\lambda \cdot V$ counts bins whose utilization exceeds the limit. Density is classically shaped during global placement [17], [18], and prior refinement operators in this flow left the term untouched: no operator could see it, so no operator could trade timing against it. We price it directly, jointly with exact timing. For each violating bin, the operator builds an eviction chain, relocating flip-flops from the bin to legal sites outside it. The chain is priced as a unit, $\alpha \sum \Delta TNS + \lambda \Delta V$, and committed only if the total is negative; otherwise every eviction in the chain is reverted. Chain-level commit-or-revert matters because a violation clears only when the bin drops below the limit, so partial evictions pay timing cost for no penalty reduction.

D. Interleaved Anytime Framework

The operators run as interleaved rounds: relocation, critical-pair swap, bit re-pairing, and structural rebanking, repeated twice, with density repair closing the schedule. Relocation and critical-pair swap are the two positional operators inherited from the reference flow, retained as move generators but re-priced by the oracle. Timing-driven relocation moves a flip-flop to a legalized site near its slack-weighted driver centroid and is accepted only on exact TNS improvement (at fixed power and area; the bin term is left to the closing density stage, as scoped above). Critical-pair swap exchanges the positions of two same-footprint cells; because the footprints match, the move is cost-free in power, area, and legality, and acceptance

reduces to the sign of the exact timing delta. Neither operator is new; what changes here is that their accept decisions are quoted by the oracle of Section III rather than by a one-hop proxy. Interleaving matters because merges create new bit re-pairing opportunities and repaired bit assignments expose new profitable merges; on one hidden case the interleaved schedule beat the same operators run as a single pass by 185K weighted cost units. Each operator exits its round early once it converges, returning unused budget to the schedule. Because all structural commits are monotone in the official cost and the schedule closes with the density stage, the framework is anytime in practice: interrupting it at any point yields a legal solution, and in all recorded runs the official score descends monotonically across stage boundaries (Fig. 2).

One repair to the inherited flow determines the move space all operators see. The flow tested a per-cell boolean as a placement-liveness predicate when selecting refinement candidates. Auditing the flag's writers showed it is a work-queue marker for the banking stage, cleared when a cell enters clustering and set only on banking's own commit paths; cells placed by the final legalizer keep the stale value forever. On one hidden case the flag silently excluded 2,024 of 6,309 physical flip-flops (32%) from every refinement operator, including all 2,000 two-bit cells, which is the entire two-bit exchange space of Section IV-A. We replace every flag test with membership in the placement map, the same structure the solution writer and the legality checkers treat as ground truth. Refinement then operates on the complete move space by construction.

V. EXPERIMENTAL RESULTS

A. Setup

We evaluated on all seven benchmarks of the ICCAD 2024 CAD Contest Problem B [4], [5]: three released cases and four hidden cases, spanning designs with millions of instances and weight profiles that price timing, power, area, and density very differently. All reported scores come from the official preliminary evaluator run on the final solution file, and every reported solution passed the evaluator's legality check plus the contest sanity and placement checkers. The flow is implemented in C++ with OpenMP on top of the strengthened open-source front end [20] described in Section II; all refinement deltas below are measured against that strengthened base, not the stock public flow.

All results use a single unified configuration: one environment recipe, zero per-case hyperparameters. The recipe was fixed by development on the same seven cases (no-regression sweeps over all seven), so like all published results on this benchmark it is selected in-sample; unlike them, no knob differs between cases, and it beat every per-case-tuned configuration this flow had previously produced. We will release the seven solution files with the final version so every score is independently checkable. Experiments ran on a dual Intel Xeon E5-2630 v4 server (2.2 GHz, 40 logical cores, 94 GB RAM) using 8 threads. Because every operator runs to its convergence fixpoint rather than to a wall-clock cut,

results are reproducible: six of the seven cases produce byte-identical solutions across all three repeats; on Hidden 4 one accept whose gain sits at floating-point noise flips between repeats, moving the final score by $2 \times 10^{-4} \%$ (std/mean).

B. Main Results

Table I compares against every evaluator-comparable published result: the contest top-3 per-case costs, the contest winner’s published method (LBR, a two-page late-breaking result [6], self-reported average ratio 0.988 against first place), and the DATE 2026 flow [10] (self-reported 0.2% better than LBR on average). Under one unified baseline, ours is the only method that beats the top-3 best on every case, and it also beats *both* published methods head to head on all seven cases: by 2.0% on Case 2 and 20.9% on Hidden 2 against LBR, and by 4.3% and 12.3% against the DATE 2026 flow. The composite is robust to its largest term: the geometric mean is 0.949, and excluding Hidden 2 entirely still yields 0.986, ahead of every published per-case-tuned result. Hidden 2’s outsized margin is itself mechanistic: on this power-dominated case, proxy-priced flows leave $2 \times 2b \rightarrow 4b$ merges unexploited that exact pricing captures safely. TIMBER [7] also reports on these cases, but under its own min–max normalization and against a baseline re-run whose bin-violation counts differ from the official contest standings, so its scores cannot be placed in this table; its published strengths are runtime and memory rather than the contest cost metric.

C. Ablation

Table II isolates each contribution by enabling the operators cumulatively on the three most cost-sensitive cases; the base row is the strengthened proxy-priced front end of Section II with the entire refinement layer disabled, so the deltas attribute to refinement only. Attribution matches the per-case cost structure. Exact batch re-pairing is the largest single lever everywhere (-2.7% to -8.6%). Repairing the inherited move-space exclusion (an audit finding rather than an algorithmic contribution) and structural rebanking pay on the two cases whose residual cost sits in banking structure (Case 2, Hidden 2), with rebanking alone worth -1.4% and -2.5% respectively. Density pricing acts exactly where overfilled bins exist: it moves Hidden 1 by -0.4% and is a verified no-op on Case 2 and Hidden 2, whose bin counts are already legal. No stage regresses any case.

A controlled counterfactual isolates pricing itself: rerunning the full flow with the rebanking operator unchanged except that each accept is quoted by the inherited one-hop proxy (identical candidates, staging, and surrounding operators) degrades every case — $+0.15\%$ on Case 2, $+0.90\%$ on Hidden 1, $+0.41\%$ on Hidden 2. Two observations follow. Exact pricing wins uniformly even inside an otherwise identical architecture; and the staged, monotone commit structure independently contains the damage that historically compounded when proxy pricing drove in-flow merging (Sec. II). Pricing fidelity and commit architecture are separable safeguards, and the historical cascades required losing both.

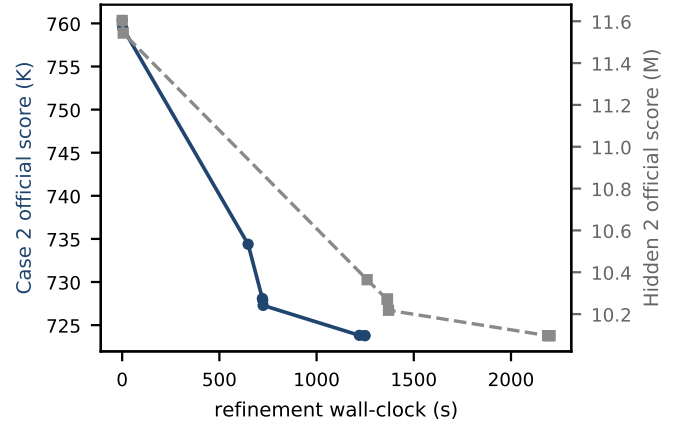


Fig. 2. Anytime behavior: official evaluator score versus refinement wall-clock, from evaluator runs on stage-boundary dumps. Case 2 (left axis, solid) and Hidden 2 (right axis, dashed).

D. Anytime Behavior

Figure 2 plots the official evaluator score, measured by running the evaluator binary on solutions dumped at every stage boundary, against refinement wall-clock on the two cases with the largest headroom. The descent is monotone across all 63 checkpoints on all seven cases.

E. Runtime

Per-case wall times appear in Table I. Convergence early-exit adapts effort automatically: cases whose operators reach fixpoints quickly finish in under four minutes, TNS-bound cases spend where descent continues, and the seven-case suite completes in about three hours on one server. Bit re-pairing dominates the budget (66–96% of refinement time); rebanking takes 4–108 s per case; density repair completes in under 0.1 s everywhere and is a no-op on the three cases with no overfilled bins. Because the framework is anytime, the runtime–quality trade can be quoted at any budget. Capping all refinement budgets to a 1–2.5 minute class (median 120 s wall) yields composite 0.966; a 3–8 minute class yields 0.958; the full-convergence configuration reaches 0.953. Every point on that curve beats the best published composite (0.979); at roughly LBR’s own runtime class this flow is already 2.5 points ahead. The base flow itself completes in 21–36 s, comparable to the contest entries; the budget beyond that is spent because the framework evaluates millions of exactly-priced moves that faster flows never attempt (measured on Hidden 2: parallel screening prices about 4,000 structural candidates per second at 8 threads; an exact trial-apply costs ~ 12 ms). For reference, the DATE 2026 flow finishes each case in about three seconds at composite 0.979 (2.5 GHz i7-11700 [10]), LBR spends up to 77 s at 0.991 (3 GHz EPYC 7313, 16 cores [6]), and we spend minutes per case on an older 2.2 GHz server to reach 0.953.

VI. CONCLUSION

We presented a post-placement MBFF refinement framework in which every accept decision is priced by an evaluator-

TABLE I

OFFICIAL EVALUATOR SCORES ON ALL SEVEN PROBLEM B CASES (LOWER IS BETTER). CONTEST TOP-3 PER-CASE COSTS AS RE-EVALUATED IN [10]; LBR SCORES AS PUBLISHED (FOUR SIGNIFICANT DIGITS) [6]. RATIOS AND COMPOSITES (AVERAGE PER-CASE RATIO) ARE COMPUTED AGAINST THE SAME TOP-3-BEST BASELINE FOR ALL METHODS. THE TWO PUBLISHED BASELINES DIFFER SLIGHTLY (E.G., CASE 2 FIRST PLACE: 748,000 IN [6] VS. 744,231 IN [10]); WE USE THE STRICTER, MAKING OUR MARGINS CONSERVATIVE.

Case	Top-3 best	LBR [6]	DATE'26 [10]	Ours	Δ vs top-3	T (s)
Case 1	739,235,861	738,800,000	736,602,486	734,766,838	−0.60%	199
Case 2	744,231	738,400	756,559	723,780	−2.75%	1,216
Case 3	727,971,140	728,800,000	727,757,372	726,236,872	−0.24%	2,582
Hidden 1	31,507,917	31,270,000	30,946,401	30,199,301	−4.15%	203
Hidden 2	13,408,414	12,760,000	11,510,067	10,097,053	−24.70%	2,202
Hidden 3	55,941,500	55,920,000	55,973,041	55,789,826	−0.27%	1,787
Hidden 4	728,497,353	726,900,000	727,798,594	726,154,906	−0.32%	2,619
Composite	1.000	0.991	0.979	0.953		

TABLE II

CUMULATIVE ABLATION, OFFICIAL EVALUATOR SCORE (LOWER IS BETTER).

Configuration	Case 2	Hidden 1	Hidden 2
Front end (no refinement)	761,159	30,615,486	11,614,120
+ exact batch re-pairing	740,251	30,336,791	10,619,091
+ move-space repair	733,903	30,335,019	10,356,851
+ structural rebanking	723,780	30,327,025	10,097,053
+ density-term pricing	723,780	30,199,301	10,097,053

exact, side-effect-free incremental timing oracle. Exact pricing converts structural rebanking from a cascading liability into strictly monotone improvement, raises batch commit throughput by an order of magnitude, and brings the density penalty into the same accept rule as timing. A single configuration beat the best contest top-3 entry on all seven ICCAD 2024 Problem B benchmarks, reaching a composite ratio of 0.953, against 0.979–0.988 for the best previously published methods [6], [10].

Three limitations bound these claims. First, all evidence comes from one benchmark family and its official cost function; behavior under other cost formulations is untested. Second, the timing model is the contest’s ideal-clock formulation, so clock-network effects lie outside both the scoring and our oracle. Third, the refinement stage is anytime; under the generous budgets used here all operators reach convergence fixpoints and runs are reproducible (six of seven cases byte-identical over three repeats), but tighter deadlines would reintroduce truncation variance.

Future work extends the move space, under the same exact-pricing discipline, to debank and eject moves and to richer target bit widths.

REFERENCES

- [1] Y.-T. Chang, C.-C. Hsu, M. P.-H. Lin, Y.-W. Tsai, and S.-F. Chen, “Post-placement power optimization with multi-bit flip-flops,” in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2010.
- [2] Y.-T. Shyu, J.-M. Lin, C.-P. Huang, C.-W. Lin, Y.-Z. Lin, and S.-J. Chang, “Effective and efficient approach for power reduction by using multi-bit flip-flops,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 21, no. 4, pp. 624–635, 2013.
- [3] A. B. Kahng, J. Li, and L. Wang, “Improved flop tray-based design implementation for power reduction,” in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2016.
- [4] S.-W. Yang, J.-W. Hsu, T.-W. Lee, T.-H. Chen, and C. C.-F. Shen, “2024 ICCAD CAD contest problem B: Power and timing optimization using multibit flip-flop,” in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024, invited contest paper.
- [5] S.-Y. Fang, Y.-Y. Liu, C.-K. Cheng, and T.-M. Tseng, “Overview of 2024 CAD contest at ICCAD,” in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.
- [6] C.-Y. Li, C.-C. Su, Z.-W. Chen, S.-H. Chen, and Y.-W. Chang, “Late breaking results: Multi-objective multi-bit flip-flop placement considering pre-placed cells,” in *Proc. Design Automation Conference (DAC)*, 2025.
- [7] A. Das Sarma, S. Jiang, W.-L. Lee, T.-Y. Ho, and T.-W. Huang, “TIMBER: A fast algorithm for timing and power optimization using multi-bit flip-flops,” in *Proc. Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2026.
- [8] A. B. Kahng, B. Kundu, and D. Thumathy, “Scalable flip-flop clustering using divide and conquer for capacitated k-means,” in *Proc. Great Lakes Symposium on VLSI (GLSVLSI)*, 2024.
- [9] Y.-Y. Chen, H.-Y. Wu, I. H.-R. Jiang, C.-H. Tsai, and C.-C. Wu, “Slack redistributed register clustering with mixed-driving strength multi-bit flip-flops,” in *Proc. International Symposium on Physical Design (ISPD)*, 2024.
- [10] H.-Y. Chen, Y.-R. Lin, M. P.-H. Lin, and H.-M. Chen, “Optimizing multibit flip-flop banking via agile in-placement PPA co-optimization,” in *Proc. Design, Automation and Test in Europe (DATE)*, 2026.
- [11] T. Gong, W. Ruan, D. Tan, Z. He, Z. Lin, and J. Chen, “Multi-bit flip-flop based timing and power optimization under advanced technology nodes,” in *Proc. IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025.
- [12] P. Liao, S. Liu, Z. Chen, W. Lv, Y. Lin, and B. Yu, “DREAMPlace 4.0: Timing-driven global placement with momentum-based net weighting,” in *Proc. Design, Automation and Test in Europe (DATE)*, 2022.
- [13] D. Lim and H. Park, “Timing-driven detailed placement with unsupervised graph learning,” in *Proc. Design, Automation and Test in Europe (DATE)*, 2025.
- [14] C. Roman-Vicharra, J. Chen, and J. Hu, “Flip-flop centric incremental placement for simultaneous timing and clock network power optimization,” in *Proc. ACM/IEEE Workshop on Machine Learning for CAD (MLCAD)*, 2024.
- [15] T.-W. Huang and M. D. F. Wong, “OpenTimer: A high-performance timing analysis tool,” in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2015.
- [16] T.-W. Huang, G. Guo, C.-X. Lin, and M. D. F. Wong, “OpenTimer v2: A new parallel incremental timing analysis engine,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 40, no. 4, pp. 776–789, 2021.
- [17] J. Lu, P. Chen, C.-C. Chang, L. Sha, D. J.-H. Huang, C.-C. Teng, and C.-K. Cheng, “ePlace: Electrostatics-based placement using fast fourier transform and Nesterov’s method,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 20, no. 2, 2015.

- [18] T.-C. Chen, Z.-W. Jiang, T.-C. Hsu, H.-C. Chen, and Y.-W. Chang, "NTUplace3: An analytical placer for large-scale mixed-size designs with preplaced blocks and density constraints," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 27, no. 7, pp. 1228–1240, 2008.
- [19] R. K. Congram, C. N. Potts, and S. L. van de Velde, "An iterated dynasearch algorithm for the single-machine total weighted tardiness scheduling problem," *INFORMS Journal on Computing*, vol. 14, no. 1, pp. 52–67, 2002.
- [20] coherent17, "2024-ICCAD-Problem-B: open-source contest flow," <https://github.com/coherent17/2024-ICCAD-Problem-B>, 2024, third-party baseline implementation.